TD-Learning

Abstract

Temporal-Difference (TD) Learning is a fundamental concept in reinforcement learning that combines ideas from Monte Carlo methods and dynamic programming. It allows for the estimation of value functions based on raw experiences without requiring a model of the environment's dynamics. This topic covers the relation of TD-learning to dynamic programming and Monte Carlo methods and introduces TD(0) as the simplest form of TD-learning, sketching also the general TD(n) case. It then explores key algorithms such as SARSA and Q-Learning.

Learning Objectives

- Relate TD-learning to DP and MC methods
- Understand the TD-Update and its relation to the Bellmann equation
- Understand and reproduce TD algorithms
- Explore n-step TD-learning
- Understand and reproduce SARSA and Q-Learning

Literature

Primary Reference

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Full PDF (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)

- **Ch. 6.1-6.3 (p. 141-151)**: TD Learning, TD(0), advantages over MC/DP
- **Ch. 6.4-6.5 (p. 151-156)**: Sarsa, Q-learning
- **Ch. 7.1 (p. 163-181)**: n-step TD methods

Supplementary Materials

Eysenbach, B. (2025). *COS435/ECE 433: Introduction to RL.* Princeton University.

- Lecture 9: Learning Value Functions (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-9-learning-value-functions.pdf) (TD target, Sarsa)
- Lecture 10: Q-Learning (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-10-q-learning.pdf) (Off-policy guarantees)

Relation to Dynamic Programming and MC Methods

Overview of Temporal-Difference Learning

Central idea of TD-learning

> **TD Learning:** Combines Monte Carlo and dynamic programming

- Bootstrapping: Recursive Value function updates (Bellmann) as in DP.
- Direct learning from raw experience, no model of environment dynamics required, as in MC.

Dynamic Programming (DP) - Model-based with bootstrapping:

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$$

- Uses **full model** $p(s',r|s,a)$ (environment dynamics)
- **Bootstraps** using current estimate $V(s')$
- Sweeps over all states

Monte Carlo (MC) - Model-free, sample-based:

$$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$$

where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$ (actual sampled return)

- Uses **sampled experience** (no dynamics model needed)
- Waits for **complete episode** to compute G_t
- No bootstrapping - uses actual returns

Comparison with Monte Carlo Methods

MC methods require episode completion

- Requires full episode completion
- Estimates based on episode returns

TD methods update incrementally

1. Update after each step
2. Uses observed reward and next state estimate
3. Learning occurs before episode ends

Comparison with Dynamic Programming

DP requires a model

- Requires full model of environment
- Less practical for real-world tasks
- Involves reward and state transition probabilities

TD methods are model-free

- Model-free learning
- Directly from raw experience
- Broad applicability

Generalized Policy Iteration

DP, MC and TD-Learning are all special cases of Generalized Policy Iteration.

The GPI Process:

- **Evaluation** (orange): $V^{\pi_n} \leftarrow$ consistent with π_n
- **Improvement** (green): $\pi_{n+1} \leftarrow$ greedy w.r.t. V^{π_n}
- **Convergence**: Iterate until π^*, V^*

GPI in Dynamic Programming

DP Characteristics:

- **Requires model**: Full knowledge of $p(s', r|s, a)$
- **Sweeps**: Updates all states synchronously or asynchronously
- **Bellman equations**: Uses expected values over all transitions

---

GPI in Monte Carlo Methods

MC Characteristics:

- **Model-free**: Learn from experience (episodes)
- **Sample averaging**: $Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s,a)}[G_t - Q(s, a)]$
- **Complete episodes**: Must wait until termination to compute G_t
- **High variance**: Full returns accumulate noise

GPI in TD, DP, and MC

- Framework: GPI
- Interaction: Policy & Value Functions
- Methods: DP, MC, TD
- Goal: Optimality

GPI's two interacting processes

1. Value Function: Predicts returns
2. Policy Improvement: Based on value function
3. Convergence: Toward optimality

GPI in Temporal-Difference Learning

TD Characteristics:

- **Model-free**: Learn from experience like MC
- **Bootstrapping**: $V(s) \leftarrow V(s) + \alpha[\underbrace{R + \gamma V(s')}_{\text{TD target}} - V(s)]$
- **Online updates**: Update after every step (no episode completion needed)
- **Lower variance**: Single reward R vs full return G_t
- **Biased but efficient**: Uses estimate $V(s')$ instead of true value

The Best of Both Worlds:

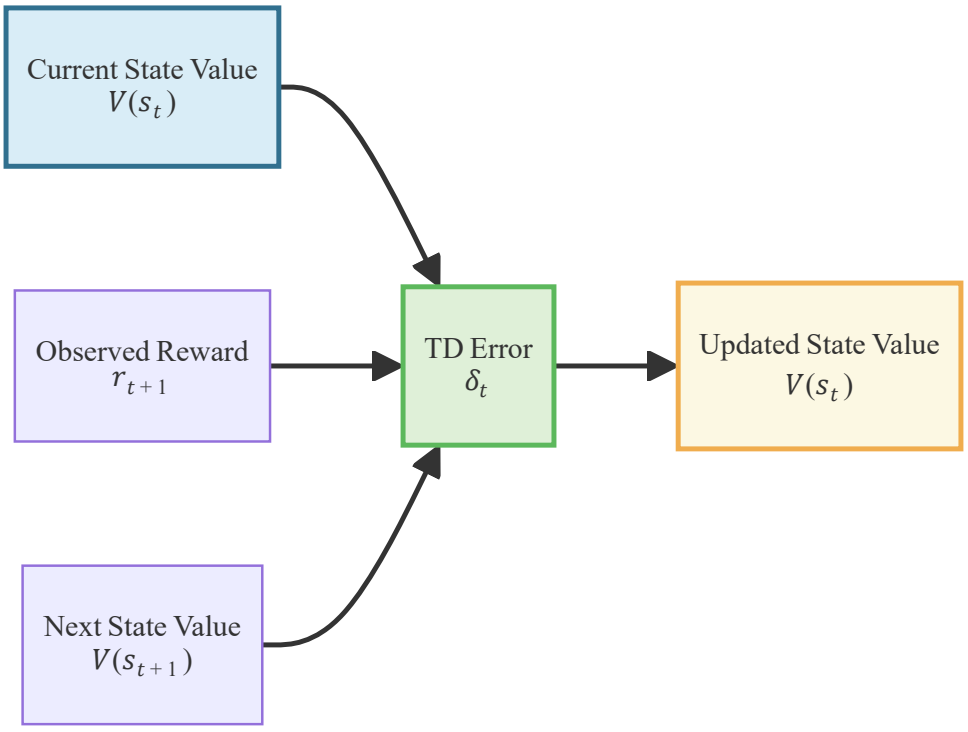
- ✅ Model-free (like MC)
- ✅ Bootstrapping (like DP)
- ✅ Online learning (unlike both MC and DP)

In [1]:

```
# This code demonstrates bootstrapping in TD-learning using a sim
ple example.
# It simulates an environment where an agent learns the value of
states
# by updating estimates based on other learned estimates, illustr
ating the concept of bootstrapping.

import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Initialize parameters
alpha = 0.1  # Step-size parameter
gamma = 0.9  # Discount factor
num_states = 5
num_episodes = 10

# Initialize value function arbitrarily
V = np.zeros(num_states)

# Simulate a simple environment
for episode in range(num_episodes):
    state = np.random.randint(0, num_states)  # Start at a random
state
    while True:
        # Simulate taking an action and receiving a reward
        next_state = (state + 1) % num_states
        reward = np.random.rand()  # Random reward

        # TD(0) update: V(S) <- V(S) + alpha * [R + gamma * V(S')
- V(S)]
        # This line demonstrates bootstrapping by updating V(S) u
sing V(S')
        V[state] += alpha * (reward + gamma * V[next_state] - V[s
tate])

        # Move to the next state
        state = next_state

        # Break the loop if a terminal condition is met (e.g., af
ter a fixed number of steps)
        if np.random.rand() < 0.1:  # Randomly end the episode
            break

# Print the learned value function
print("Learned Value Function:", V)
```

```
Learned Value Function: [1.28940055 1.23430646 1.25554731 1.25357
959 1.2534741 ]
```

TD Error and its Significance



Definition of TD error

> **TD Error:** Difference between estimated and updated state values

TD error's role in learning

- Drives updates in RL algorithms
- Measures value estimate error per time step

Why does this recursive "bootstrapping" approach work?

1. The Bellman Equation for State Values (see also L2)

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s], \quad \text{for all } s \in \mathcal{S} \end{aligned}$$

2. Deriving the TD Update Rule

Starting from the Bellman equation, we want to update our estimate $V(s_t)$ toward the true value $V^\pi(s_t)$.

Step 1: The Bellman equation tells us the true value

$$V^\pi(s_t) = \mathbb{E}_\pi[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t]$$

Step 2: Use a sample-based estimate (TD target)

Instead of computing the full expectation, we use a single sample (s_t, r_{t+1}, s_{t+1}) from experience:

$$\text{TD target} = r_{t+1} + \gamma V(s_{t+1})$$

This is an approximation of $V^\pi(s_t)$ using our current estimate $V(s_{t+1})$ for the next state.

Step 3: Compute the error (TD error)

The difference between our target and current estimate:

$$\delta_t = \underbrace{r_{t+1} + \gamma V(s_{t+1})}_{\text{TD target}} - \underbrace{V(s_t)}_{\text{current estimate}}$$

Step 4: Update toward the target (stochastic gradient descent)

We move our estimate in the direction of the error, scaled by learning rate α:

$$\boxed{V(s_t) \leftarrow V(s_t) + \alpha\delta_t = V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]}$$

This is the **TD(0) update rule**.

The TD(n) family

Why TD(0)?

The update rule we've seen is called **TD(0)** (not just "TD") because:

- **"0" indicates how many steps we "delay bootstrapping"**
- **TD(0)**: Uses **1 step** $\rightarrow r_{t+1} + \gamma V(s_{t+1})$ (1 reward, then bootstrap)
- **TD(1)**: Uses **2 steps** $\rightarrow r_{t+1} + \gamma r_{t+2} + \gamma^2 V(s_{t+2})$ (2 rewards, then bootstrap)
- **TD(n)**: Uses $n+1$ **steps** $\rightarrow r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n r_{t+n+1} + \gamma^{n+1} V(s_{t+n+1})$ ($n+1$ rewards, then bootstrap)
- **TD(∞)**: Looks all the way to episode end $\rightarrow$ equivalent to **Monte Carlo** (full return G_t)

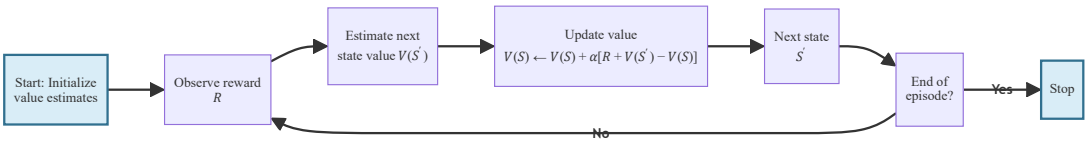
The TD family:

$$\begin{aligned}
\mathrm{TD}(0): &\quad V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)] \\
\mathrm{TD}(1): &\quad V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma r_{t+2} + \gamma^2 V(s_{t+2}) - V(s_t)] \\
\mathrm{TD}(n): &\quad V(s_t) \leftarrow V(s_t) + \alpha[\sum_{k=0}^{n} \gamma^k r_{t+k+1} + \gamma^{n+1} V(s_{t+n+1}) - V(s_t)] \\
\mathrm{TD}(\infty): &\quad V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)] \quad (\text{Monte Carlo})
\end{aligned}$$

Trade-off:

- **Lower** n (e.g., TD(0)): Lower variance, higher bias, faster updates
- **Higher** n: Higher variance, lower bias, approaches MC

The TD(0) Algorithm



- It is called TD "0" because we look ahead only one step: $V(S')$.
- We could look further (TD(n)), where $n = \infty$ is equivalent to MC. (Reinforcement Learning: An Introduction, Second Edition, Richard S. Sutton, Andrew G. Barto, 2018, p. 141-146)

Algorithm: TD(0) Prediction for estimating V

(Sutton & Barto, 2018, Section 6.1, p. 120)

$$
\begin{aligned}
&\textbf{Input: } \pi \text{ (policy to evaluate)} \\
&\textbf{Output: } V^{\pi}(s) \text{ (value function for all states)} \\
&\\
&\textbf{1.} \text{ Initialize } V(s) \text{ arbitrarily for all non-terminal states } s \in \mathcal{S}, \text{ and } V(\text{terminal}) = 0 \\
&\textbf{2.} \text{ Set step-size parameter } \alpha \in (0, 1] \\
&\textbf{3. Repeat for each episode:} \\
&\quad \textbf{4.} \text{ Initialize state } s \\
&\quad \textbf{5. Loop for each step of episode:} \\
&\quad\quad \textbf{6.} \ a \leftarrow \text{action given by } \pi \text{ for } s \\
&\quad\quad \textbf{7.} \text{ Take action } a, \text{ observe } r, s' \\
&\quad\quad \textbf{8.} \ V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)] \\
&\quad\quad \textbf{9.} \ s \leftarrow s' \\
&\quad \textbf{until } s \text{ is terminal}
\end{aligned}
$$

Key Components:

- **Step 1**: Initialize V(s) arbitrarily for all states, but **V(terminal) = 0** (required)
- **Step 8**: TD(0) update rule - uses current estimate V(s') including V(terminal)=0
- **Loop termination**: "until s is terminal" - after s ← s', check if s is terminal
- **Bootstrapping**: Uses current estimate V(s') instead of waiting for actual return
- **Online learning**: Updates happen immediately after each transition

In [2]:
```
# TD(0) Prediction Algorithm Implementation
# Based on Sutton & Barto, Section 6.1

import numpy as np
import gymnasium as gym
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import update_value_function_plot, plot_lea
rning_curves
import time

def td_prediction(env_name='FrozenLake-v1',
                  policy_fn=None,
                  num_episodes=1000,
                  alpha=0.1,
                  gamma=0.95,
                  initial_value='zeros',
                  show_animation=False,
                  plot_interval=100):
    """
    TD(0) Prediction algorithm for estimating V^π.

    Args:
        env_name: Gymnasium environment name
        policy_fn: Policy function π(s) -> a. If None, uses rando
m policy
        num_episodes: Number of episodes to run
        alpha: Step-size parameter (learning rate), 0 < α ≤ 1
        gamma: Discount factor
        initial_value: 'zeros' or 'random' initialization
        show_animation: If True, show value function updates
        plot_interval: Update visualization every N episodes

    Returns:
        value_function: Estimated V^π(s)
        episode_stats: Dictionary with learning statistics
    """
    # Step 1: Initialize environment
    hidden_env = gym.make(env_name, is_slippery=False, render_mod
e=None,
                          max_episode_steps=50, reward_schedule=
(1, 0, 0))
    rendered_env = gym.make(env_name, is_slippery=False, render_m
ode='human',
                            max_episode_steps=50, reward_schedule=
(1, 0, 0)) if show_animation else None

    # Step 1: Initialize V(s) for all s ∈ S⁺, arbitrarily except
V(terminal) = 0
    # (Sutton & Barto, Section 6.1, p. 120)
    num_states = hidden_env.observation_space.n

    if initial_value == 'zeros':
        value_function = np.zeros(num_states)
    elif initial_value == 'random':
```

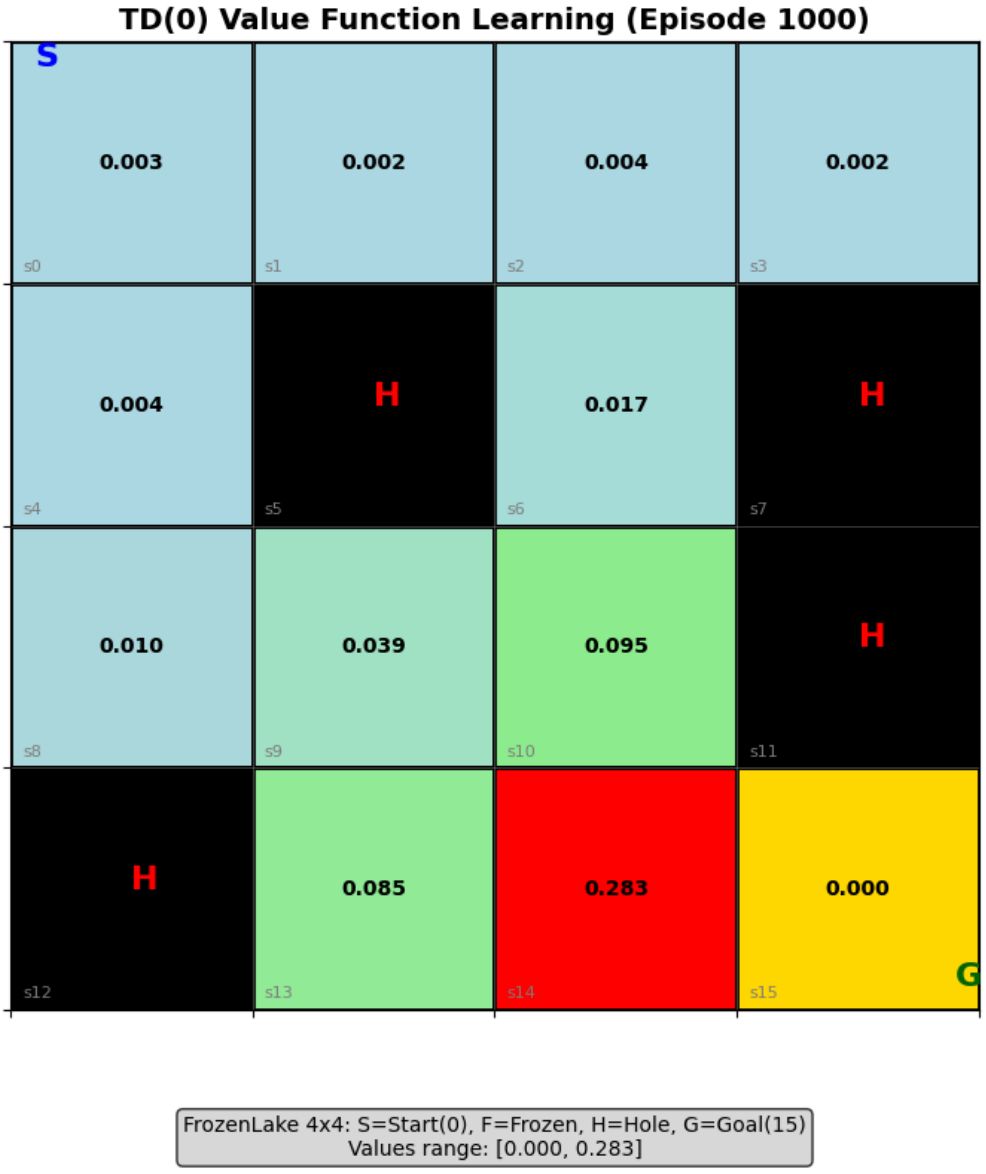
```
        value_function = np.random.rand(num_states)
        # Ensure terminal states have value 0 (required by algori
thm)
        # For FrozenLake, terminal states are holes and goal
        if env_name == 'FrozenLake-v1':
            terminal_states = [5, 7, 11, 12, 15]  # Holes + goal
            value_function[terminal_states] = 0.0
    else:
        value_function = np.ones(num_states) * 0.5

    # Default random policy if none provided
    if policy_fn is None:
        def policy_fn(state):
            """Simple random policy for demonstration"""
            return np.random.choice(hidden_env.action_space.n)

    # Statistics tracking
    episode_stats = {
        'episode_returns': [],
        'episode_lengths': [],
        'value_changes': []
    }

    # Show initial state if animating
    if show_animation:
        update_value_function_plot(value_function,
                                   "TD(0) Value Function",
                                   episode=0)
        time.sleep(1)

    # Step 3: Loop for each episode
    for episode in range(num_episodes):
        # Use rendered environment occasionally for visualization
        if show_animation and episode % plot_interval == 0:
            print(f"Episode {episode}/{num_episodes}")
            current_env = rendered_env if show_animation else hid
den_env
        else:
            current_env = hidden_env

        # Step 4: Initialize S (start state)
        state = current_env.reset()[0]
        done = False
        total_reward = 0
        steps = 0
        V_old = value_function.copy()

        # Step 5: Loop for each step of episode
        while not done:
            # Step 6: A ← action given by π for S
            action = policy_fn(state)

            # Step 7: Take action A, observe R, S'
            next_state, reward, done, truncated, info = current_e
nv.step(action)
            total_reward += reward
            steps += 1
```

```
            # Step 8: V(S) ← V(S) + α[R + γV(S') - V(S)]
            # Note: When S' is terminal, V(S') = 0 by initializat
ion (Step 1)
            # For environments like FrozenLake, terminal states e
xist in state space
            # For environments like Random Walk boundaries, must
handle explicitly
            td_target = reward + gamma * value_function[next_stat
e]
            td_error = td_target - value_function[state]
            value_function[state] += alpha * td_error

            # Step 9: S ← S'
            state = next_state

            if truncated:
                done = True

        # Track episode statistics
        episode_stats['episode_returns'].append(total_reward)
        episode_stats['episode_lengths'].append(steps)
        max_change = np.max(np.abs(value_function - V_old))
        episode_stats['value_changes'].append(max_change)

        # Show progress with animation
        if show_animation and (episode % plot_interval == 0 or ep
isode == num_episodes - 1):
            update_value_function_plot(value_function,
                                       "TD(0) Value Function Learn
ing",
                                       episode=episode+1)
            if episode < num_episodes - 1:
                time.sleep(0.5)

    # Print final results
    print(f"\nCompleted {num_episodes} episodes with α = {alpha},
γ = {gamma}")
    print("Estimated Value Function V^π(s):")
    for state in range(num_states):
        print(f"State {state}: V = {value_function[state]:.4f}")

    print(f"\nEnvironment Description:")
    if env_name == 'FrozenLake-v1':
        print("FrozenLake 4x4 grid (deterministic)")
        print("Goal: Reach the frisbee (state 15) from start (sta
te 0)")
        print("Holes at states: 5, 7, 11, 12")

    # Calculate additional statistics
    avg_episode_length = np.mean(episode_stats['episode_length
s'])
    success_rate = np.mean([r > 0 for r in episode_stats['episode
_returns']])

    print(f"\nStatistics:")
    print(f"Average episode length: {avg_episode_length:.1f}")
```

```
    print(f"Success rate: {success_rate:.3f}")
    print(f"Final max value change: {episode_stats['value_change
s'][-1]:.6f}")

    # Close environments
    hidden_env.close()
    if rendered_env:
        rendered_env.close()

    return value_function, episode_stats
```

In [3]:

```
# Run TD(0) Prediction on FrozenLake
# Using a random policy (default when policy_fn=None)

print("🎬 Running TD(0) Prediction with Random Policy")
print("=" * 50)

print("\n📊 TD(0) Prediction on FrozenLake")
value_function_td, stats_td = td_prediction(
    env_name='FrozenLake-v1',
    policy_fn=None,  # Random policy
    num_episodes=1000,
    alpha=0.1,
    gamma=0.95,
    initial_value='zeros',
    show_animation=True,
    plot_interval=100
)

print(f"\n🎯 Key Results:")
print(f"Value of goal state (15): {value_function_td[15]:.4f}")
print(f"Value of start state (0): {value_function_td[0]:.4f}")
print(f"States with highest values: {np.argsort(value_function_t
d)[-5:]}")
```


TD(0) Value Function Learning (Episode 1000)
S
0.003
0.002
0.004
0.002
s0
s1
s2
s3
0.004
H
0.017
H
s4
s5
s6
s7
0.010
0.039
0.095
H
s8
s9
s10
s11
H
0.085
0.283
0.000
G
s12
s13
s14
s15
FrozenLake 4x4: S=Start(0), F=Frozen, H=Hole, G=Goal(15)
Values range: [0.000, 0.283]

```
Completed 1000 episodes with α = 0.1, γ = 0.95
Estimated Value Function V^π(s):
State 0: V = 0.0029
State 1: V = 0.0017
State 2: V = 0.0044
State 3: V = 0.0017
State 4: V = 0.0036
State 5: V = 0.0000
State 6: V = 0.0173
State 7: V = 0.0000
State 8: V = 0.0100
State 9: V = 0.0388
State 10: V = 0.0949
State 11: V = 0.0000
State 12: V = 0.0000
State 13: V = 0.0852
State 14: V = 0.2835
State 15: V = 0.0000

Environment Description:
FrozenLake 4x4 grid (deterministic)
Goal: Reach the frisbee (state 15) from start (state 0)
Holes at states: 5, 7, 11, 12

Statistics:
Average episode length: 7.9
Success rate: 0.012
Final max value change: 0.000189

🎯 Key Results:
Value of goal state (15): 0.0000
Value of start state (0): 0.0029
States with highest values: [ 6  9 13 10 14]
```

Convergence Properties of TD Prediction

Are TD Methods Sound?

It's convenient to learn one estimate from the next without waiting for actual outcomes—but does this guarantee convergence to the correct answer?

Happily, yes! ✓

TD(0) Convergence Guarantee:

For any fixed policy π, TD(0) has been proved to converge to the true value function $V^\pi(s)$:

- **In the mean** for a constant step-size parameter if it is sufficiently small (i.e., the expected value converges: $\mathbb{E}[V_t(s)] \to V^\pi(s)$, though individual estimates may fluctuate)
- **With probability 1** if the step-size parameter decreases appropriately (i.e., satisfying $\sum_{t=1}^{\infty} \alpha_t = \infty$ and $\sum_{t=1}^{\infty} \alpha_t^2 < \infty$, such as $\alpha_t = 1/t$)

Most convergence proofs apply to the tabular case, but some also apply to general linear function approximation.

[(Sutton & Barto, 2018, Section 6.2, p. 146-151) (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)](https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)

Empirical Convergence: TD(0) vs Monte Carlo

Comparison of TD(0) with Monte Carlo:

Feature	Monte Carlo	TD(0)
Update timing	End of episode	Every step
Target	G_t (actual return)	$R + \gamma V(S')$ (estimated return)
Variance	High	Low
Bias	Unbiased	Biased (initially)
Episode requirement	Must complete	No requirement

Empirical Convergence Speed:

In practice, TD(0) has usually been found to converge **faster** than constant-step-size Monte Carlo methods on stochastic tasks.

[(Sutton & Barto, 2018, p. 146-151) (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)](https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)

Why is TD(0) Faster?

1. **Online learning**: TD(0) updates after every step, not waiting for episode completion
   - MC must wait until end of episode to update any state
   - TD can learn from incomplete episodes and continuing tasks
2. **Lower variance**: Bootstrapping from $V(S')$ reduces variance compared to full return G_t
   - MC target G_t depends on all future rewards (high variance)
   - TD target $R + \gamma V(S')$ depends on one reward plus current estimate (lower variance)
   - Lower variance enables faster, more stable convergence
3. **Immediate propagation**: Value updates propagate backwards immediately
   - In episodic tasks, TD updates states as they're visited
   - MC must complete episode before any state values are updated
   - TD can learn from terminal rewards much faster
4. **Better sample efficiency**: Each transition provides learning signal
   - MC uses one data point (return) per episode
   - TD uses one data point per step within episode
   - More frequent updates lead to faster convergence

Trade-off: TD(0) is biased (bootstraps from estimates) but lower variance; MC is unbiased but higher variance. In practice, the bias-variance trade-off favors TD for convergence speed.

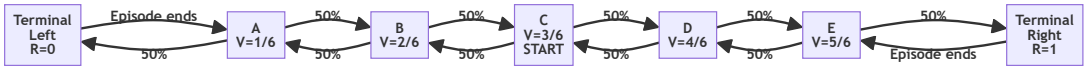
Example: Random Walk (Sutton & Barto, Example 6.2)

This classic example compares TD(0) and Monte Carlo on a simple Markov Reward Process (MRP):

Setup:

- **States**: A, B, C, D, E (plus two terminal states on left and right)
- **Start**: Always start in state C (center)
- **Transitions**: Move left or right with equal probability (50/50)
- **Rewards**: +1 when terminating on the right, 0 otherwise
- **True values**: $V(A) = \frac{1}{6}, V(B) = \frac{2}{6}, V(C) = \frac{3}{6}, V(D) = \frac{4}{6}, V(E) = \frac{5}{6}$

The true value of each state is the probability of eventually terminating on the right from that state.



In [4]:
```
# Random Walk Example: Comparing DP, TD(0), and Monte Carlo
# Based on Sutton & Barto, Example 6.2, p. 125

import numpy as np
import matplotlib.pyplot as plt
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import (generate_random_walk_episode,
                              random_walk_dp_solution,
                              plot_random_walk_results,
                              print_random_walk_summary)

def random_walk_comparison(num_states=5, start_state=2, alpha=0.
1,
                           num_episodes=100, num_runs=100, gamma=
1.0,
                           initial_value=0.5, show_plots=True):
    """
    Compare DP, TD(0), and Monte Carlo on Random Walk task.

    Args:
        num_states: Number of non-terminal states (default: 5 for
A-E)
        start_state: Starting state (default: 2 = C, center)
        alpha: Learning rate for TD and MC
        num_episodes: Episodes per run
        num_runs: Number of independent runs for averaging
        gamma: Discount factor (1.0 for undiscounted)
        initial_value: Initial value estimate for all states
        show_plots: Whether to display plots

    Returns:
        results: Dictionary with all method results and statistic
s
    """
    # Set random seed for reproducibility
    np.random.seed(42)

    # State labels
    states = ['A', 'B', 'C', 'D', 'E'][:num_states]

    # True values: probability of terminating on the right
    true_values = np.array([(i+1)/(num_states+1) for i in range(n
um_states)])

    # ===== Method 1: Dynamic Programming (Exact Solution) =====
    V_dp, dp_iterations = random_walk_dp_solution(num_states)

    # ===== Method 2: TD(0) - ONLINE LEARNING =====
    # TD(0) updates after EVERY step, not after episodes
    rms_errors_td = np.zeros((num_runs, num_episodes))

    for run in range(num_runs):
        V_td = np.ones(num_states) * initial_value
```

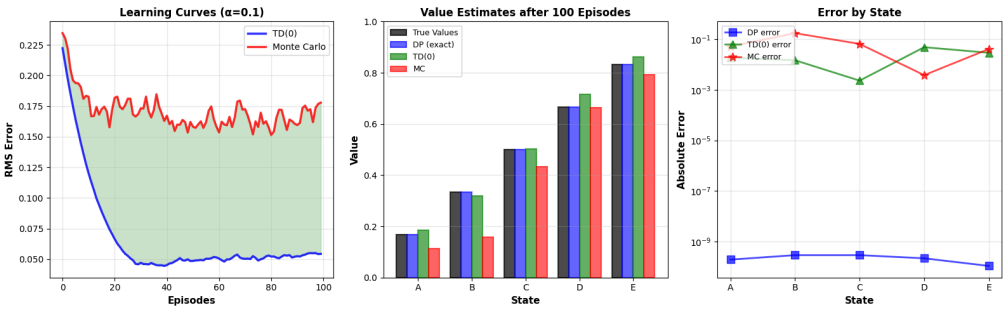
```
        for episode in range(num_episodes):
            # Initialize state S for this episode
            state = start_state

            # Loop for each step of episode (online updates)
            while True:
                # Move left or right with equal probability
                if np.random.rand() < 0.5:
                    next_state = state - 1
                else:
                    next_state = state + 1

                # Check for terminal states
                if next_state < 0:  # Terminal left
                    reward = 0
                    next_value = 0  # V(terminal) = 0
                    is_terminal = True
                elif next_state >= num_states:  # Terminal right
                    reward = 1
                    next_value = 0  # V(terminal) = 0
                    is_terminal = True
                else:
                    reward = 0
                    next_value = V_td[next_state]  # V(S')
                    is_terminal = False

                # TD(0) update: V(S) ← V(S) + α[R + γV(S') - V
(S)]
                # This happens IMMEDIATELY after the transition
                td_target = reward + gamma * next_value
                V_td[state] += alpha * (td_target - V_td[state])

                # Move to next state
                if is_terminal:
                    break
                state = next_state

            # Calculate RMS error after each episode
            rms_errors_td[run, episode] = np.sqrt(np.mean((V_td -
true_values)**2))

    avg_rms_td = np.mean(rms_errors_td, axis=0)

    # ===== Method 3: Monte Carlo - OFFLINE LEARNING =====
    # MC waits for complete episodes and uses actual returns
    rms_errors_mc = np.zeros((num_runs, num_episodes))

    for run in range(num_runs):
        V_mc = np.ones(num_states) * initial_value

        for episode in range(num_episodes):
            episode_states, episode_rewards = generate_random_wal
k_episode(start_state, num_states)

            # Calculate return (undiscounted, so just sum)
            G = sum(episode_rewards)
```

```
            # MC update for all states in episode
            for state in episode_states:
                V_mc[state] += alpha * (G - V_mc[state])

            # Calculate RMS error after each episode
            rms_errors_mc[run, episode] = np.sqrt(np.mean((V_mc -
true_values)**2))

    avg_rms_mc = np.mean(rms_errors_mc, axis=0)

    # ===== Get final estimates from a single run =====
    # TD(0): Online updates step-by-step
    V_td_final = np.ones(num_states) * initial_value
    for episode in range(num_episodes):
        state = start_state
        while True:
            if np.random.rand() < 0.5:
                next_state = state - 1
            else:
                next_state = state + 1

            if next_state < 0:
                reward = 0
                next_value = 0
                is_terminal = True
            elif next_state >= num_states:
                reward = 1
                next_value = 0
                is_terminal = True
            else:
                reward = 0
                next_value = V_td_final[next_state]
                is_terminal = False

            td_target = reward + gamma * next_value
            V_td_final[state] += alpha * (td_target - V_td_final
[state])

            if is_terminal:
                break
            state = next_state

    # MC: Offline updates using episodes
    V_mc_final = np.ones(num_states) * initial_value
    for episode in range(num_episodes):
        episode_states, episode_rewards = generate_random_walk_ep
isode(start_state, num_states)
        G = sum(episode_rewards)
        for state in episode_states:
            V_mc_final[state] += alpha * (G - V_mc_final[state])

    # Print summary
    print_random_walk_summary(V_dp, V_td_final, V_mc_final, true_
values,
                              avg_rms_td, avg_rms_mc, dp_iteration
s,
                              num_episodes, num_runs, alpha)
```

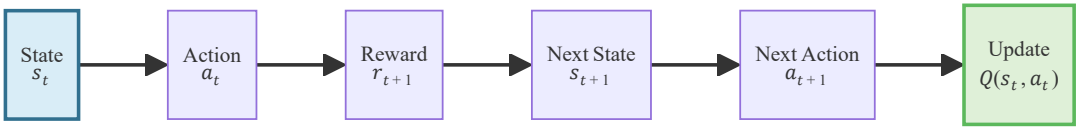
```
    # Visualization
    if show_plots:
        plot_random_walk_results(avg_rms_td, avg_rms_mc, V_dp, V_
td_final,
                                 V_mc_final, true_values, alpha, n
um_episodes, states)

    # Return all results
    return {
        'V_dp': V_dp,
        'V_td': V_td_final,
        'V_mc': V_mc_final,
        'true_values': true_values,
        'avg_rms_td': avg_rms_td,
        'avg_rms_mc': avg_rms_mc,
        'dp_iterations': dp_iterations
    }
```

In [5]:
```
# Run Random Walk comparison with default parameters
results = random_walk_comparison(
    num_states=5,
    start_state=2,
    alpha=0.1,
    num_episodes=100,
    num_runs=100,
    gamma=1.0,
    initial_value=0.5,
    show_plots=True
)
```

```
================================================================
=====
Random Walk: DP, TD(0), and Monte Carlo Comparison
================================================================
=====

DP Solution:
  Converged in 76 iterations
  Values: [0.16666667 0.33333333 0.5        0.66666667 0.8333333
3]

TD(0) (α=0.1, 100 episodes, 100 runs):
  Final RMS error: 0.0543
  Values: [0.18569048 0.31844506 0.50232226 0.71566716 0.8628666
4]

Monte Carlo (α=0.1, 100 episodes, 100 runs):
  Final RMS error: 0.1778
  Values: [0.11403034 0.1568833  0.43389286 0.66283433 0.7918491
]

================================================================
=====
Result: TD(0) achieves 0.0543 RMS error vs MC 0.1778
================================================================
=====
```



On-policy TD control: SARSA



SARSA Definition:

SARSA is an on-policy TD control algorithm that estimates the action-value function Q for the current policy by updating Q-values using the sequence (state, action, reward, next state, next action), hence the name SARSA.

(Sutton & Barto, 2018, Section 6.4, p. 129-130)

Update Rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

where a_{t+1} is chosen according to the current policy (typically ε-greedy).

Key Properties:

- On-policy: Evaluates and improves the policy being followed
- Online learning: Updates after each transition
- Uses quintuple: $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$

Algorithm: Pseudocode for SARSA

(Sutton & Barto, 2018, Section 6.4, p. 130)

Input: policy π (e.g., ε-greedy w.r.t. Q)
Output: $Q(s, a)$ (action-value function)

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
1. Initialize $Q(s, a)$ arbitrarily for all non-terminal states $s \in \mathcal{S}$ and actions $a \in \mathcal{A}(s)$, a

2. Loop for each episode:
  3. Initialize s
  4. Choose a from s using policy derived from Q (e.g., ε-greedy)
  5. Loop for each step of episode:
    6. Take action a, observe r, s'
    7. Choose a' from s' using policy derived from Q (e.g., ε-greedy)
    8. $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
    9. $s \leftarrow s'; \quad a \leftarrow a'$
  until s is terminal

Key Steps:

- **Step 4 & 7**: Choose actions using ε-greedy policy derived from current Q
- **Step 8**: SARSA update uses the actual next action a' taken by the policy
- **On-policy**: Both behavior and target policies are the same (ε-greedy)
- **Converges**: To optimal policy as ε decreases and sufficient exploration occurs

In [6]:

```python
# SARSA Algorithm Implementation
# Based on Sutton & Barto, Section 6.4, p. 130

import numpy as np
import gymnasium as gym
import matplotlib.pyplot as plt
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import plot_q_values_frozen_lake
import time

def sarsa_control(env_name='FrozenLake-v1',
                 num_episodes=5000,
                 alpha=0.1,
                 gamma=0.99,
                 epsilon=0.1,
                 random_init=True,
                 show_animation=False,
                 plot_interval=500):
    """
    SARSA: On-Policy TD Control algorithm.
    Following Sutton & Barto, Section 6.4, p. 130

    Args:
        env_name: Gymnasium environment name
        num_episodes: Number of episodes to run
        alpha: Step size (learning rate), 0 < α ≤ 1
        gamma: Discount factor
        epsilon: Small ε > 0 for ε-greedy policy
        random_init: If True, initialize Q randomly; if False, us
e initial_q_value
        show_animation: If True, show learning progress
        plot_interval: Update visualization every N episodes

    Returns:
        Q: Learned action-value function
        policy: Learned policy (greedy w.r.t. Q)
        stats: Dictionary with learning statistics
    """
    # Create environment
    env = gym.make(env_name, is_slippery=False, render_mode=None,
                   max_episode_steps=100, reward_schedule=(1, 0,
0))

    # Step 1: Initialize Q(s,a) for all s ∈ S⁺, a ∈ A(s) arbitrar
ily
    # except Q(terminal, ·) = 0
    num_states = env.observation_space.n
    num_actions = env.action_space.n
    if random_init:
        Q = np.random.rand(num_states, num_actions)
    else:
        Q = np.zeros((num_states, num_actions))

    # Set terminal states Q-values to 0
```

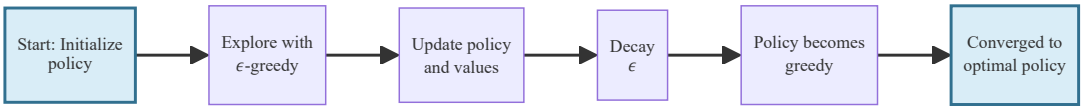
```
    if env_name == 'FrozenLake-v1':
        terminal_states = [5, 7, 11, 12, 15]  # Holes + goal
        Q[terminal_states, :] = 0.0

    def epsilon_greedy_policy(state):
        """ε-greedy policy derived from Q"""
        if np.random.rand() < epsilon:
            return np.random.choice(num_actions)
        else:
            return np.argmax(Q[state])

    # Statistics tracking (for visualization, not in pseudocode)
    stats = {
        'episode_returns': [],
        'episode_lengths': [],
        'q_value_changes': []
    }

    # Show initial Q-values
    if show_animation:
        plot_q_values_frozen_lake(Q, "SARSA Q-values (Initial)",
0)
        time.sleep(5)

    # Step 2: Loop for each episode
    for episode in range(num_episodes):
        # Step 3: Initialize S
        state, _ = env.reset()

        # Step 4: Choose A from S using policy derived from Q (ε-
greedy)
        action = epsilon_greedy_policy(state)

        total_reward = 0
        steps = 0
        Q_old = Q.copy()
        done = False

        # Step 5: Loop for each step of episode
        while not done:
            # Step 6: Take action A, observe R, S'
            next_state, reward, done, truncated, info = env.step
(action)
            total_reward += reward
            steps += 1

            # Step 7: Choose A' from S' using policy derived from
Q (ε-greedy)
            next_action = epsilon_greedy_policy(next_state)

            # Step 8: Q(S,A) ← Q(S,A) + α[R + γQ(S',A') - Q(S,A)]
            td_target = reward + gamma * Q[next_state, next_actio
n]
            td_error = td_target - Q[state, action]
            Q[state, action] += alpha * td_error

            # Step 9: S ← S'; A ← A'
```

```
            state = next_state
            action = next_action

            if truncated:
                done = True

        # Track statistics (for visualization)
        stats['episode_returns'].append(total_reward)
        stats['episode_lengths'].append(steps)
        max_q_change = np.max(np.abs(Q - Q_old))
        stats['q_value_changes'].append(max_q_change)

        # Show progress
        if show_animation and (episode % plot_interval == 0 or ep
isode == num_episodes - 1):
            print(f"\nEpisode {episode}/{num_episodes}, ε={epsilo
n:.4f}, Return={total_reward}, Steps={steps}")
            plot_q_values_frozen_lake(Q, f"SARSA Q-values", episo
de+1)
            if episode < num_episodes - 1:
                time.sleep(0.3)

    # Extract greedy policy from Q
    policy = np.argmax(Q, axis=1)

    # Print results
    print(f"\nCompleted {num_episodes} episodes")
    avg_last_100 = np.mean(stats['episode_returns'][-100:])
    print(f"Average return (last 100 episodes): {avg_last_100:.4
f}")
    print(f"Success rate (last 100 episodes): {np.sum(np.array(st
ats['episode_returns'][-100:]) > 0)/100:.2%}")

    # Close environment
    env.close()

    return Q, policy, stats
```

In [7]:
```
# Run SARSA on FrozenLake
Q_sarsa, policy_sarsa, stats_sarsa = sarsa_control(
    env_name='FrozenLake-v1',
    num_episodes=300,
    alpha=0.1,
    gamma=0.99,
    epsilon=0.1,
    show_animation=True,
    random_init=True,
    plot_interval=2
)
```

```
Completed 300 episodes
Average return (last 100 episodes): 0.8600
Success rate (last 100 episodes): 86.00%
```

Convergence Properties of SARSA



SARSA Convergence Guarantee:

SARSA converges with probability 1 to the optimal policy and action-value function under the following conditions:

- All state-action pairs are visited infinitely often
- The policy becomes greedy in the limit (e.g., by decaying ϵ in ϵ-greedy policies)

(Sutton & Barto, 2018, Section 6.4, p. 129-130)

Q-Learning: Off-Policy TD control

Motivation: SARSA's Limitation

SARSA is an on-policy method: it must follow the same policy it's trying to improve. With a fixed exploration rate ϵ, SARSA is forced to:

- Continue exploring even after learning a good policy
- Update Q-values based on exploratory (suboptimal) actions
- Never learn the true optimal policy when $\epsilon > 0$

Solution: Separate Behavior from Learning

Q-Learning addresses this by decoupling:

- **Behavior policy**: Explores the environment (e.g., ϵ-greedy)
- **Target policy**: Learns the optimal policy (greedy: $\max_{a'} Q(s', a')$)

On-Policy vs Off-Policy:

Method	Policy Type	Update Mechanism
SARSA	On-policy	Uses current policy's action a'
Q-Learning	Off-policy	Uses $\max_{a'} Q(s', a')$

This allows Q-Learning to learn the optimal policy while still exploring, making it more sample-efficient and converging to the true optimal policy regardless of the behavior policy.

Q-Learning Algorithm

(Sutton & Barto, 2018, Section 6.5, p. 131)

Algorithm: Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Initialize $Q(s, a)$ arbitrarily for all non-terminal states $s \in \mathcal{S}$ and actions $a \in \mathcal{A}(s)$, and set $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

- Initialize s
- Loop for each step of episode:
  - Choose a from s using policy derived from Q (e.g., ϵ-greedy)
  - Take action a, observe r, s'
  - $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
  - $s \leftarrow s'$
- until s is terminal

Question: From the MC lecture we know that off-policy learning involves a behavior policy b and a target policy π. What is what in this case?

In [8]:

```
# Q-Learning Algorithm Implementation
# Based on Sutton & Barto, Section 6.5, p. 131

import numpy as np
import gymnasium as gym
import matplotlib.pyplot as plt
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import plot_q_values_frozen_lake
import time

def q_learning_control(env_name='FrozenLake-v1',
                       num_episodes=5000,
                       alpha=0.1,
                       gamma=0.99,
                       epsilon=0.1,
                       random_init=True,
                       show_animation=False,
                       plot_interval=500):
    """
    Q-Learning: Off-Policy TD Control algorithm.
    Following Sutton & Barto, Section 6.5, p. 131

    Args:
        env_name: Gymnasium environment name
        num_episodes: Number of episodes to run
        alpha: Step size (learning rate), 0 < α ≤ 1
        gamma: Discount factor
        epsilon: Small ε > 0 for ε-greedy behavior policy
        random_init: If True, initialize Q randomly; if False, in
itialize with zeros
        show_animation: If True, show learning progress
        plot_interval: Update visualization every N episodes

    Returns:
        Q: Learned action-value function
        policy: Learned policy (greedy w.r.t. Q)
        stats: Dictionary with learning statistics
    """
    # Create environment
    env = gym.make(env_name, is_slippery=False, render_mode=None,
                   max_episode_steps=100, reward_schedule=(1, 0,
0))

    # Step 1: Initialize Q(s,a) for all s ∈ S⁺, a ∈ A(s) arbitrar
ily
    # except Q(terminal, ·) = 0
    num_states = env.observation_space.n
    num_actions = env.action_space.n
    if random_init:
        Q = np.random.rand(num_states, num_actions)
    else:
        Q = np.zeros((num_states, num_actions))

    # Set terminal states Q-values to 0
```

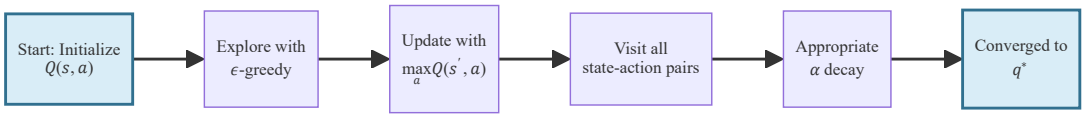
```
    if env_name == 'FrozenLake-v1':
        terminal_states = [5, 7, 11, 12, 15]  # Holes + goal
        Q[terminal_states, :] = 0.0

    def epsilon_greedy_policy(state):
        """ε-greedy behavior policy derived from Q"""
        if np.random.rand() < epsilon:
            return np.random.choice(num_actions)
        else:
            return np.argmax(Q[state])

    # Statistics tracking (for visualization, not in pseudocode)
    stats = {
        'episode_returns': [],
        'episode_lengths': [],
        'q_value_changes': []
    }

    # Show initial Q-values
    if show_animation:
        plot_q_values_frozen_lake(Q, "Q-Learning Q-values (Initia
l)", 0)
        time.sleep(5)

    # Step 2: Loop for each episode
    for episode in range(num_episodes):
        # Step 3: Initialize S
        state, _ = env.reset()

        total_reward = 0
        steps = 0
        Q_old = Q.copy()
        done = False

        # Step 4: Loop for each step of episode
        while not done:
            # Step 5: Choose A from S using policy derived from Q
(e.g., ε-greedy)
            action = epsilon_greedy_policy(state)

            # Step 6: Take action A, observe R, S'
            next_state, reward, done, truncated, info = env.step
(action)
            total_reward += reward
            steps += 1

            # Step 7: Q(S,A) ← Q(S,A) + α[R + γ max_a Q(S',a) - Q
(S,A)]
            # Key difference from SARSA: uses max_a instead of A'
            td_target = reward + gamma * np.max(Q[next_state])
            td_error = td_target - Q[state, action]
            Q[state, action] += alpha * td_error

            # Step 8: S ← S'
            state = next_state

            if truncated:
```

```
                done = True

        # Track statistics (for visualization)
        stats['episode_returns'].append(total_reward)
        stats['episode_lengths'].append(steps)
        max_q_change = np.max(np.abs(Q - Q_old))
        stats['q_value_changes'].append(max_q_change)

        # Show progress
        if show_animation and (episode % plot_interval == 0 or ep
isode == num_episodes - 1):
            print(f"\nEpisode {episode}/{num_episodes}, ε={epsilo
n:.4f}, Return={total_reward}, Steps={steps}")
            plot_q_values_frozen_lake(Q, f"Q-Learning Q-values",
episode+1)
            if episode < num_episodes - 1:
                time.sleep(0.3)

    # Extract greedy policy from Q
    policy = np.argmax(Q, axis=1)

    # Print results
    print(f"\nCompleted {num_episodes} episodes")
    avg_last_100 = np.mean(stats['episode_returns'][-100:])
    print(f"Average return (last 100 episodes): {avg_last_100:.4
f}")
    print(f"Success rate (last 100 episodes): {np.sum(np.array(st
ats['episode_returns'][-100:]) > 0)/100:.2%}")

    # Close environment
    env.close()

    return Q, policy, stats
```

In [9]:
```
# Run Q-Learning on FrozenLake
Q_qlearning, policy_qlearning, stats_qlearning = q_learning_contr
ol(
    env_name='FrozenLake-v1',
    num_episodes=300,
    alpha=0.1,
    gamma=0.99,
    epsilon=0.1,
    show_animation=True,
    random_init=True,
    plot_interval=2
)
```

```
Completed 300 episodes
Average return (last 100 episodes): 0.8800
Success rate (last 100 episodes): 88.00%
```

Convergence Properties of Q-Learning



Q-Learning Convergence Guarantee:

Q-Learning converges with probability 1 to the optimal action-value function q^* under the following conditions:

- All state-action pairs are visited infinitely often
- The step-size parameter α is appropriately decayed

(Sutton & Barto, 2018, Section 6.5, p. 131-133)

Q-Learning and SARSA Comparison

Side-by-Side Pseudocode Comparison

SARSA (On-Policy)	**Q-Learning (Off-Policy)**
Initialize $Q(s,a)$ arbitrarily, $Q(\text{terminal},\cdot)=0$	Initialize $Q(s,a)$ arbitrarily, $Q(\text{terminal},\cdot)=0$
Loop for each episode:	**Loop** for each episode:
Initialize S	Initialize S
Choose A from S using policy derived from Q	*(no action selection)*
Loop for each step of episode:	**Loop** for each step of episode:
(no action selection)	**Choose** A from S using policy derived from Q
Take action A, observe R, S'	Take action A, observe R, S'
Choose A' from S' using policy derived from Q	*(no next action selection)*
$Q(S,A) \leftarrow Q(S,A) + \alpha[R+\gamma Q(S',A')-Q(S,A)]$	$Q(S,A) \leftarrow Q(S,A) + \alpha[R+\gamma \max_a Q(S',a) - Q(S,A)]$
$S \leftarrow S'$; $A \leftarrow A'$	$S \leftarrow S'$
until S is terminal	until S is terminal

Key Differences:

SARSA (On-Policy):

- Chooses A' before updating, uses $Q(S',A')$ in update
- **Consequence - Policy Dependence**: Learns the value of the policy being followed, including exploratory actions
- **Consequence - Convergence**: Converges to optimal policy only if the policy becomes greedy in the limit (e.g., ϵ decays to 0)
- **Consequence - Safety**: Takes into account the exploratory nature of behavior, leading to more conservative policies

Q-Learning (Off-Policy):

- Uses $\max_a Q(S',a)$ in update, separates action selection from evaluation
- **Consequence - Policy Independence**: Learns the value of the optimal policy regardless of the policy being followed
- **Consequence - Convergence**: Converges directly to Q^* (optimal action-value function) even while following an exploratory policy
- **Consequence - Optimality**: Can learn the optimal policy while behaving non-optimally, but may be more aggressive in stochastic environments

(Sutton & Barto, 2018, Section 6.4-6.5, p. 130-133)

Cliff Walking: SARSA vs Q-Learning

The Cliff Walking environment demonstrates the key behavioral difference between SARSA and Q-Learning:

Environment:

- Grid world with a cliff along the bottom row
- Start: Bottom-left corner
- Goal: Bottom-right corner
- Cliff: All cells between start and goal in bottom row
- Falling off cliff: Returns agent to start with reward of -100
- Each step: Reward of -1

Expected Behavior:

SARSA (On-Policy):

- Learns to take a **safe path** away from the cliff edge
- Takes exploratory actions into account during learning
- Results in longer but safer routes

Q-Learning (Off-Policy):

- Learns the **optimal path** right along the cliff edge
- Ignores exploratory actions in value estimates
- During exploration, frequently falls off cliff, but learns optimal policy

(Sutton & Barto, 2018, Example 6.6, p. 132)

In [10]:

```python
# Cliff Walking Implementation
# Comparing SARSA and Q-Learning behavior

import numpy as np
import gymnasium as gym
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import (plot_cliff_walking_learning_curves,
                              visualize_cliff_walking_policy)

def run_cliff_walking_comparison(num_episodes=500, alpha=0.5, gam
ma=1.0, epsilon=0.1):
    """
    Compare SARSA and Q-Learning on Cliff Walking environment.

    Args:
        num_episodes: Number of episodes to run
        alpha: Learning rate
        gamma: Discount factor
        epsilon: Exploration rate
    """

    # Create environment
    env = gym.make('CliffWalking-v1')

    num_states = env.observation_space.n
    num_actions = env.action_space.n

    # Initialize Q-tables for both algorithms
    Q_sarsa = np.zeros((num_states, num_actions))
    Q_qlearning = np.zeros((num_states, num_actions))

    def epsilon_greedy(Q, state):
        """ε-greedy policy"""
        if np.random.rand() < epsilon:
            return np.random.choice(num_actions)
        else:
            return np.argmax(Q[state])

    # Track performance
    returns_sarsa = []
    returns_qlearning = []

    print("=" * 60)
    print("SARSA Learning (On-Policy)")
    print("=" * 60)

    # SARSA Learning
    for episode in range(num_episodes):
        state, _ = env.reset()
        action = epsilon_greedy(Q_sarsa, state)
        total_reward = 0
        done = False

        while not done:
```

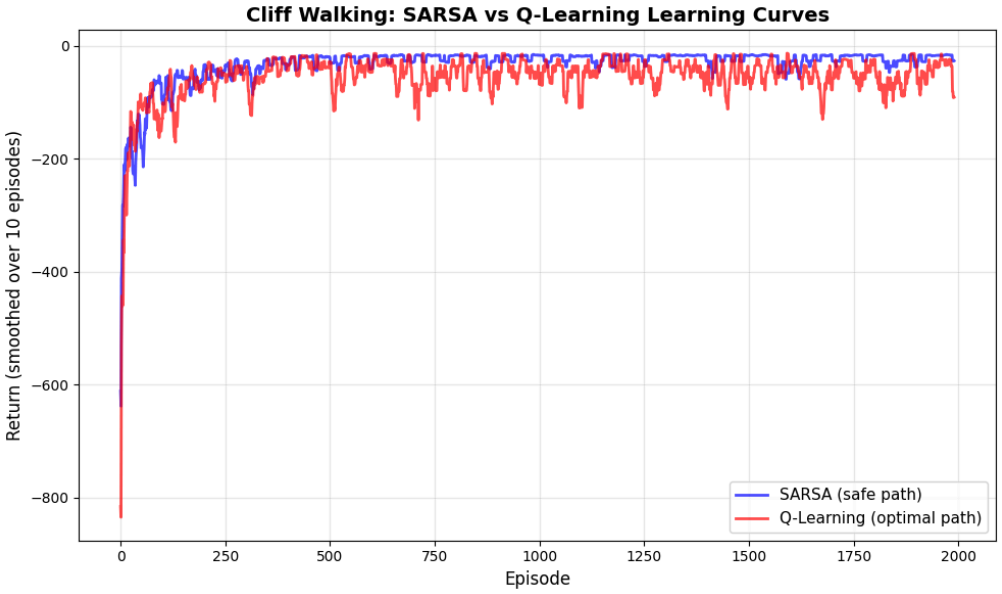
```
            next_state, reward, done, truncated, _ = env.step(act
ion)
            total_reward += reward

            # SARSA update
            next_action = epsilon_greedy(Q_sarsa, next_state)
            td_target = reward + gamma * Q_sarsa[next_state, next
_action]
            td_error = td_target - Q_sarsa[state, action]
            Q_sarsa[state, action] += alpha * td_error

            state = next_state
            action = next_action

            if truncated:
                done = True

        returns_sarsa.append(total_reward)

        if (episode + 1) % 500 == 0:
            print(f"Episode {episode + 1}/{num_episodes}")

    print("\n" + "=" * 60)
    print("Q-Learning (Off-Policy)")
    print("=" * 60)

    # Q-Learning
    for episode in range(num_episodes):
        state, _ = env.reset()
        total_reward = 0
        done = False

        while not done:
            action = epsilon_greedy(Q_qlearning, state)
            next_state, reward, done, truncated, _ = env.step(act
ion)
            total_reward += reward

            # Q-Learning update
            td_target = reward + gamma * np.max(Q_qlearning[next_
state])
            td_error = td_target - Q_qlearning[state, action]
            Q_qlearning[state, action] += alpha * td_error

            state = next_state

            if truncated:
                done = True

        returns_qlearning.append(total_reward)

        if (episode + 1) % 500 == 0:
            print(f"Episode {episode + 1}/{num_episodes}")

    env.close()

    # Plot learning curves
```

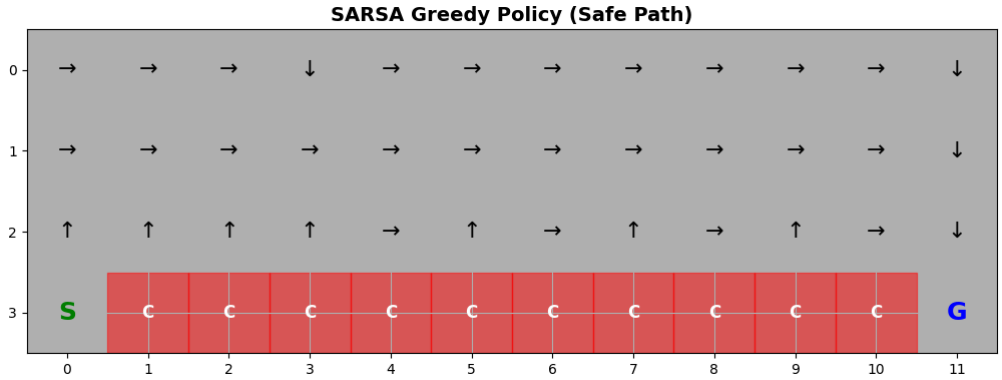
```
    plot_cliff_walking_learning_curves(returns_sarsa, returns_qle
arning, window=10)

    # Visualize learned greedy policies (π(s) = argmax_a Q(s,a))
    visualize_cliff_walking_policy(Q_sarsa, 'SARSA Greedy Policy
(Safe Path)')
    visualize_cliff_walking_policy(Q_qlearning, 'Q-Learning Greed
y Policy (Optimal Path)')

    # Print statistics
    print("\n" + "=" * 60)
    print(f"Results after {num_episodes} episodes:")
    print("=" * 60)
    print(f"SARSA average return (last 100): {np.mean(returns_sar
sa[-100:]):.2f}")
    print(f"Q-Learning average return (last 100): {np.mean(return
s_qlearning[-100:]):.2f}")
    print("\nKey Observation:")
    print("- SARSA learns SAFER path (higher return during learni
ng)")
    print("- Q-Learning learns OPTIMAL path (lower return during
learning, but shortest path)")

    return Q_sarsa, Q_qlearning, returns_sarsa, returns_qlearning
```

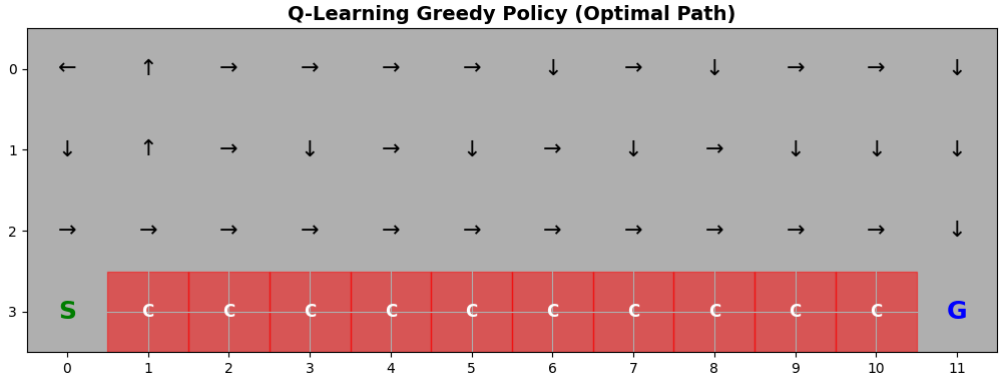
In [11]:

```
# Run the comparison
# Note: Using lower learning rate (alpha=0.1) and more episodes f
or stable convergence
Q_sarsa_cliff, Q_qlearning_cliff, returns_sarsa, returns_qlearnin
g = run_cliff_walking_comparison(
    num_episodes=2000,
    alpha=0.05,
    gamma=0.99,
    epsilon=0.1
)
```

```
============================================================
SARSA Learning (On-Policy)
============================================================
Episode 500/2000
Episode 500/2000
Episode 1000/2000
Episode 1500/2000
Episode 2000/2000

============================================================
Q-Learning (Off-Policy)
============================================================
Episode 1000/2000
Episode 1500/2000
Episode 2000/2000

============================================================
Q-Learning (Off-Policy)
============================================================
Episode 500/2000
Episode 1000/2000
Episode 1500/2000
Episode 500/2000
Episode 1000/2000
Episode 1500/2000
Episode 2000/2000
Episode 2000/2000
```








```
============================================================
Results after 2000 episodes:
============================================================
SARSA average return (last 100): -21.31
Q-Learning average return (last 100): -45.62

Key Observation:
- SARSA learns SAFER path (higher return during learning)
- Q-Learning learns OPTIMAL path (lower return during learning, b
ut shortest path)
```

Summary: Temporal-Difference Learning Methods

In this lecture, we explored TD learning methods that combine advantages of both Monte Carlo and Dynamic Programming approaches:

TD Algorithms Covered

Algorithm	Type	Update Rule	Key Characteristic
TD(0) Prediction	Value estimation	$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$	One-step bootstrapping for state values
SARSA	On-policy control	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$	Learns value of policy being followed
Q-Learning	Off-policy control	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$	Learns optimal policy regardless of behavior

Key Concepts

- **Bootstrapping**: TD methods update estimates based on other estimates (unlike MC which uses complete returns)
- **Online Learning**: Updates occur after each step, not requiring complete episodes
- **On-Policy vs Off-Policy**:
  - SARSA (on-policy): Conservative, learns safe paths, converges to policy being executed
  - Q-Learning (off-policy): Aggressive, learns optimal paths, converges to Q^*
- **TD Error**: $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ drives learning

Common Characteristics

- **Model-free**: No knowledge of environment dynamics required
- **Generalized Policy Iteration**: Interleave policy evaluation and improvement
- **Lower variance than MC**: Bootstrapping reduces variance but introduces bias
- **Work for continuing tasks**: Don't require episode termination
- **Sample efficiency**: Learn from incomplete sequences

Cliff Walking Demonstration

Our empirical comparison demonstrated:

- **SARSA**: Learns safe path avoiding cliff edge (second row from bottom)
- **Q-Learning**: Learns optimal path along cliff edge (bottom row), converging to Q^*
- Both converge to stable greedy policies with proper hyperparameters ($\alpha = 0.1$, $\gamma = 1.0$, 2000 episodes)

(Sutton & Barto, 2018, Chapter 6)

Outlook: Toward Function Approximation and Actor-Critic Methods

While TD learning methods (SARSA, Q-Learning) successfully learn value functions using tabular representations, they face fundamental limitations:

Limitations of Tabular Methods

- **State space explosion**: Q-tables become infeasible for large or continuous state spaces
- **No generalization**: Each state must be visited separately; no learning transfer between similar states
- **Memory requirements**: Storage grows linearly with state-action pairs
- **Discrete actions**: Continuous action spaces cannot be represented in tables

Next Step: Function Approximation

To scale TD methods to complex problems, we replace Q-tables with **parameterized function approximators**:

$$Q(s, a) \approx Q_\phi(s, a)$$

where ϕ represents learnable parameters of the value function approximator.

Deep Q-Networks (DQN) extend Q-Learning with:

- Neural networks to approximate $Q(s, a)$
- Experience replay for stable learning
- Target networks for convergence
- Applications to high-dimensional inputs (images, continuous states)

This allows TD methods to handle Atari games, robotics, and other complex domains.